BA305 — Final Exam · Version 1 · Solutions

Total: 65 points. Part A: 40 points · Part B: 25 points.

Part A — Concepts (Questions 1–15, 40 points)

No code in this part.

Part A: 40 points.

Question 1 — Multiple choice (2 points)

Why is the Naïve Bayes classifier called "naïve"?

- (a) It assumes that, within each class, the features are unrelated to each other.
- (b) It assumes that every class is equally common in the data.
- (c) It looks at only one feature at a time and ignores the others.
- (d) It ignores how common each class is in the training data.

> **Answer — (a).** Naïve Bayes multiplies one probability per feature, which is only right if the features are independent of each other within each class. That is rarely true (in spam, "free" and "offer" tend to appear together), hence "naïve". (b) and (d) The class shares are the first factor of each score. (c) Every feature contributes a factor.

Question 2 — Multiple choice (3 points)

Two product reviews contain exactly the same words in a different order: "not good, actually great" and "not great, actually good." A Naïve Bayes model trained on word counts scores both. What happens?

- (a) Both get the same score, because the model counts words and ignores their order.
- (b) They get different scores, because "not" changes the meaning of the word right after it.
- (c) They get different scores, because the model gives more weight to words near the start.
- (d) The model cannot score them, because the same words appear in both classes.

> **Answer — (a).** A word-count model ignores word order. Both reviews have one "not", one "good", one "actually" and one "great", so they get the same score. (b) The model does not read "not" together with the next word. (c) Position plays no part. (d) Words found in both classes are normal; each simply adds its own factor.

Question 3 — True / False (3 points)

A spam filter, a Naïve Bayes model, was trained when 10% of incoming emails were spam. Spam has since grown to 50% of incoming emails, and the filter has not been retrained. The filter will now tend to label too few emails as spam.

- True
- False

> **Answer — True.** Each class's score starts with its share in the training data. The filter still multiplies spam scores by 0.10 and normal scores by 0.90, which holds spam down now that half the emails are spam, so borderline spam gets labeled normal.

Question 4 — True / False (2 points)

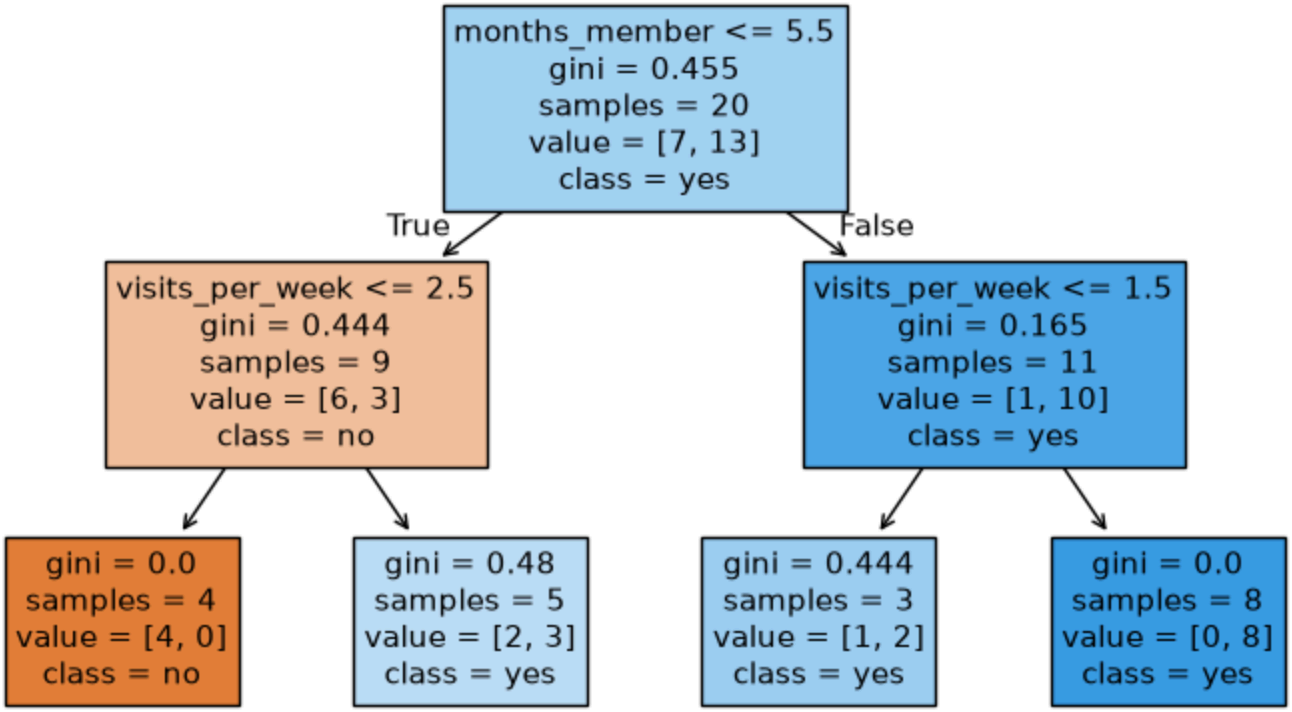
In a classification tree, a node with a Gini index of 0 contains records of only one class.

- True
- False

> **Answer — True.** Gini = $1 - \Sigma\, p_i^2$ is 0 only when one class has share 1: a pure node, the best possible value.

Question 5 — Multiple choice (2 points)

A gym fit a classification tree to predict whether a member renews (yes / no) from `visits_per_week` and `months_member`.



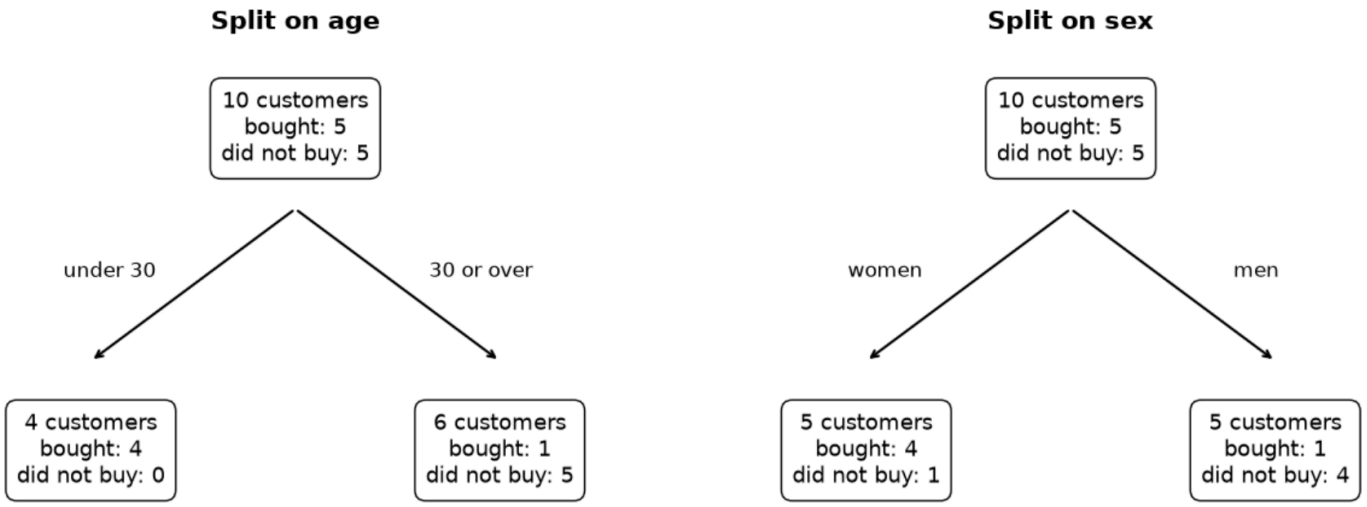
What does the tree predict for a member of 10 months who visits once a week, and how many similar training members support that prediction?

- (a) Renews; 2 of the 3 training members in that leaf renewed.
- (b) Does not renew; 1 of the 3 training members in that leaf did not renew.
- (c) Renews; all 8 training members in that leaf renewed.
- (d) Does not renew; all 4 training members in that leaf did not renew.

> **Answer — (a).** 10 months > 5.5 → right; 1 visit ≤ 1.5 → left, to the leaf [1, 2]: 1 did not renew, 2 renewed. The majority is "yes", backed by only 3 members. (b) That leaf's majority is "yes". (c) The [0, 8] leaf is for more than 1.5 visits per week. (d) The [4, 0] leaf is for members of at most 5.5 months.

Question 6 — Multiple choice (3 points)

A node holds 10 customers: 5 bought and 5 did not. The tree can split them on age or on sex, as shown below.



The tree scores each split by the number of customers it misclassifies. Which split does it choose?

- (a) Age: it misclassifies 1 customer, against 2 for sex.
- (b) Sex: its two sides hold the same number of customers, so the split is more balanced.
- (c) Sex: it misclassifies 8 customers, against 9 for age.
- (d) Neither: both splits misclassify the same number of customers.

> **Answer — (a).** Each side predicts its majority. Age: the left side is pure (0 mistakes) and the right side gets its 1 buyer wrong, so 1 mistake. Sex: 1 wrong on each side, so 2 mistakes. Fewer mistakes wins. (b) Balance is not the criterion. (c) counts each side's smaller class instead of its mistakes. (d) The counts differ.

Question 7 — Select all that apply (4 points)

Which statements about a random forest, compared with a single fully grown tree, are true? Select **all** that apply.

- (a) Each tree in the forest is trained on a random sample of the records.
- (b) The forest predicts by aggregating its trees' decisions, such as a majority vote.
- (c) Each tree in the forest is built using all of the features.
- (d) It is as easy to explain to a customer as a single tree.
- (e) It cannot overfit, however deep its trees grow.

> **Answer — (a) and (b).** (a) Each tree sees its own random sample of the records. (b) The trees' decisions are combined by majority vote (or by averaging, for numbers). (c) Each tree uses only a random subset of the features. (d) Hundreds of trees give no single set of rules to show a customer. (e) A forest overfits less than one deep tree, but it can still overfit.

Question 8 — Multiple choice (3 points)

A logistic regression predicts whether a customer renews a subscription. The coefficient of `years_as_customer` is 0.40, and e^0.40 = 1.49. The model gives one customer a probability of 0.30 of renewing. Holding the other features

fixed, what probability does the model give the same customer after one more year as a customer?

- (a) About 0.39
- (b) About 0.45
- (c) About 0.60
- (d) About 0.70

> **Answer — (a).** Odds = 0.30 / 0.70 = 0.429. One more year multiplies the odds by 1.49: 0.429 × 1.49 = 0.639. Probability = 0.639 / 1.639 = 0.39. (b) multiplies the probability instead of the odds. (c) is the logistic of 0.40 alone, ignoring where the customer started. (d) adds the coefficient to the probability.

Question 9 — True / False (3 points)

A model gives a customer odds of 0.25 of renewing a subscription. The probability that the customer renews is therefore 0.25.

- True
- False

> **Answer — False.** p = odds / (1 + odds) = 0.25 / 1.25 = 0.2. Odds of 0.25 mean 1 renewal for every 4 non-renewals, so 1 customer in 5 renews.

Question 10 — Multiple choice (2 points)

In a logistic regression, coefficients above which number mean that a feature is positively related to the outcome?

- (a) −1
- (b) 0
- (c) 0.5
- (d) 1

> **Answer — (b).** A coefficient above 0 means a higher value of the feature makes the outcome more likely; below 0, less likely. (a) and (c) Values between −1 and 0 are negative, and values between 0 and 0.5 are positive too. (d) 1 is the cutoff for exp(coefficient), not for the coefficient.

Question 11 — True / False (3 points)

Changing every activation function in a neural network from sigmoid to linear does not affect its ability to classify data that is not linearly separable.

- True
- False

Answer — False. With linear activations everywhere, the network collapses into one linear model, however many layers it has, so it can only draw a straight-line boundary. A curved boundary is possible only with non-linear activations (such as the sigmoid) in the hidden layers, and even then it is not guaranteed.

Question 12 — Select all that apply (2 points)

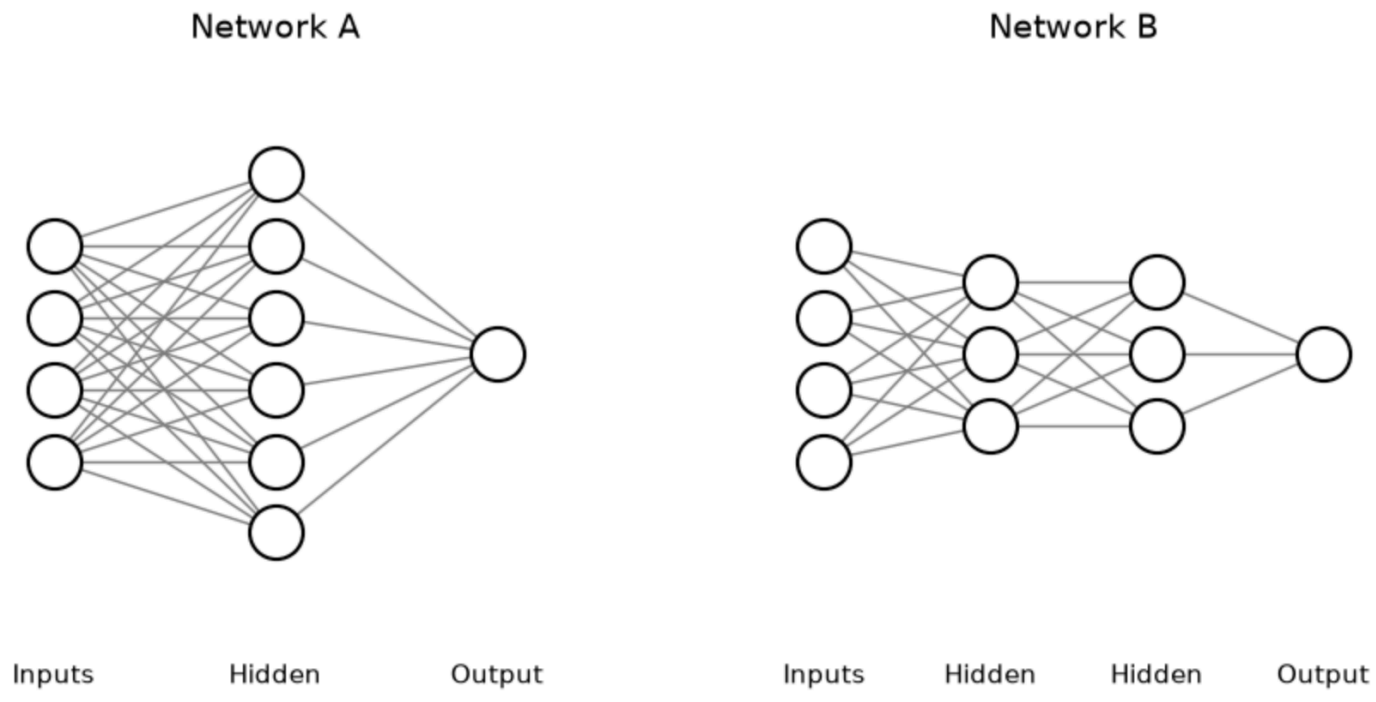
A bank must give every rejected loan applicant the main reasons for the decision. Which models make it hard to give those reasons? Select **all** that apply.

- (a) A classification tree with three levels
- (b) A logistic regression
- (c) A neural network with two hidden layers of 20 nodes each
- (d) A k-nearest neighbors model with k = 5

Answer — (c) and (d). (c) A network's decision passes through hundreds of weights and two layers, so no single number explains it. (d) k-NN has no rules or coefficients: it copies the 5 most similar applicants, mixing all features at once. (a) A three-level tree is a short list of if-then rules. (b) Each coefficient says how a feature moves the odds.

Question 13 — Multiple choice (3 points)

The two networks below have the same inputs, the same output and the same number of hidden nodes, spread over the hidden layers differently. Every node in one layer connects to every node in the next.



Both are trained on the same data. Which network is more likely to overfit?

- (a) Network A
- (b) Network B
- (c) Both are equally likely, because they have the same number of nodes
- (d) Neither, because overfitting depends only on the data, not on the network

> **Answer — (a).** More parameters (weights) give a network more room to memorize the training data. Count the connections: A has 4 × 6 + 6 × 1 = 30; B has 4 × 3 + 3 × 3 + 3 × 1 = 24. (b) More layers do not always mean more parameters. (c) Parameters sit on the connections, not the nodes. (d) The model matters as well as the data.

Question 14 — Select all that apply (3 points)

A customer table has `income` (in dollars, from 20,000 to 300,000) and `age` (in years, from 18 to 90). For which of these models do the very different ranges of the two features change the result unless you scale them? Select **all** that apply.

- (a) k-nearest neighbors
- (b) A neural network
- (c) A classification tree
- (d) A random forest
- (e) PCA

> **Answer — (a), (b) and (e).** k-NN distances, a neural network's weighted sums and PCA's variances are all dominated by income's large numbers. (c) and (d) A tree, and a forest of trees, splits one feature at a time, so rescaling only moves the thresholds.

Question 15 — Multiple choice (2 points)

A clinic uses a model to decide which patients get a follow-up test for a serious but treatable disease. The follow-up test is cheap and safe. Which of these numbers should the clinic care about most?

- (a) Recall
- (b) Precision
- (c) Accuracy
- (d) Specificity

> **Answer — (a).** Recall is the share of sick patients the model sends for a test. A missed sick patient is costly and an extra test is cheap and safe, so catch as many as possible. (b) Precision matters when false alarms are costly. (c) Accuracy can look high while missing sick patients. (d) Specificity is about the healthy patients.

Part B — Coding questions (Questions 16–23, 25 points)

The code and its output are in two separate handouts, **Handout B1** and **Handout B2**. Each question names the exhibits it uses. You cannot run the code.

Part B: 25 points. Total: 65 points.

Case B1 — Long Beach Animal Shelter (use Handout B1)

The animal shelter of Long Beach, California, recorded every animal it took in from 2017 to 2024. The data is in a DataFrame named `shelter`, one row per animal:

Column	Meaning
`kind`	Kind of animal: cat, dog, wild (wildlife) or other
`sex`	Sex of the animal
`age_years`	Age on arrival, in years
`condition`	Condition on arrival: normal, ill, injured or other
`intake_type`	How the animal arrived, such as stray or owner surrender
`outcome_type`	How its stay ended, such as adoption, transfer to a rescue group, or euthanasia
`alive`	1 if the animal left the shelter alive, 0 if not

On the day an animal arrives, the shelter wants to know whether it is at risk of not leaving alive, so it can give those animals medical care and send them to foster homes first.

Exhibit 1

```
# Number of animals, and the fraction that left the shelter alive.
print('Animals:', len(shelter))
shelter['alive'].value_counts(normalize=True).round(3)
```

```
Animals: 26047

alive
1    0.82
0    0.18
Name: proportion, dtype: float64
```

Exhibit 2

```
# The first few animals.
shelter.head()
```

	kind	sex	age_years	condition	intake_type	outcome_type	alive
0	dog	Female	10.002740	ill	stray	euthanasia	0
1	cat	Female	4.002740	injured	stray	rescue	1
2	other	Neutered	1.501370	normal	stray	adoption	1
3	other	Unknown	0.167123	other	wildlife	rescue	1
4	cat	Unknown	0.084932	other	stray	rescue	1

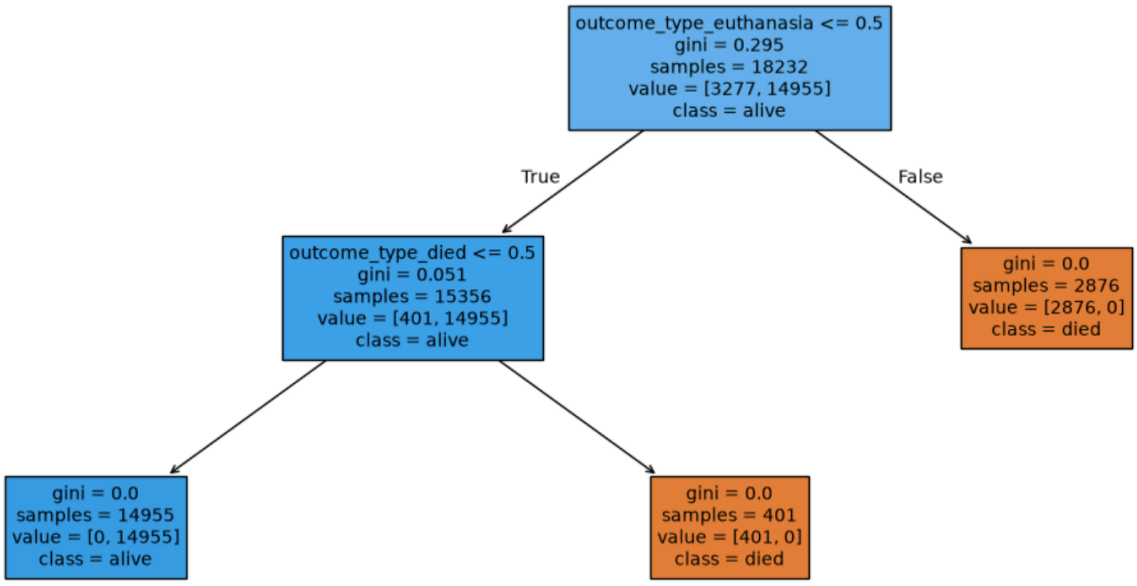
Question 16 — True / False (2 points)

Exhibit 3

```
# Take a first subset of the features. Hold out 30% for testing.
X = pd.get_dummies(shelter[['kind', 'condition', 'intake_type', 'outcome_type']], dtype=int)
y = shelter['alive']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)

tree_1 = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
print('Test accuracy:', round(accuracy_score(y_test, tree_1.predict(X_test)), 3))
plt.figure(figsize=(16, 7))
plot_tree(tree_1, feature_names=list(X.columns), class_names=['died', 'alive'], filled=True,
fontsize=10);
```

```
Test accuracy: 1.0
```



The shelter wants to flag at-risk animals on the day they arrive, so it can give them medical care and send them to foster homes first. A test accuracy of 1.0 on animals it never saw shows that the shelter can use `tree_1`, the tree in Exhibit 3, to do that.

- True
- False

> **Answer — False.** Every split uses `outcome_type` (euthanasia or died means not alive): that is the answer written another way, and it is unknown on the day an animal arrives. This is leakage, so the test accuracy of 1.0 cannot be reproduced at intake.

Question 17 — Multiple choice (4 points)

Exhibit 4

```
# Fraction of each kind among the animals that died (0) and among those that left alive (1).
table = pd.crosstab(shelter['kind'], shelter['alive'], normalize='columns')
table.loc['Total'] = table.sum()
table.round(2)
```

alive	0	1
kind		
cat	0.61	0.50
dog	0.15	0.42
other	0.16	0.08
wild	0.09	0.01
Total	1.00	1.00

Exhibit 5

```
# Fraction of each condition among the animals that died (0) and among those that left alive (1).
table = pd.crosstab(shelter['condition'], shelter['alive'], normalize='columns')
table.loc['Total'] = table.sum()
table.round(2)
```

alive	0	1
condition		
ill	0.26	0.07
injured	0.28	0.07
normal	0.13	0.57
other	0.33	0.28
Total	1.00	1.00

In both tables, column 0 holds the animals that did not leave the shelter alive and column 1 the animals that did. For example, 0.42 in the `dog` row of column 1 means that 42% of the animals that left alive were dogs.

A Naïve Bayes model is trained on all the animals in `shelter`, with `kind` and `condition` as its only features. A wild animal arrives injured. Which calculation and prediction are correct?

- (a) Alive: 0.82 × 0.01 × 0.07 = 0.00057; did not: 0.18 × 0.09 × 0.28 = 0.00454. It predicts the animal will not leave alive.
- (b) Alive: 0.01 × 0.07 = 0.00070; did not: 0.09 × 0.28 = 0.02520. It predicts the animal will not leave alive.
- (c) Alive: 0.82 × 0.01 × 0.07 = 0.00057. This is far below 0.5, so it predicts the animal will not leave alive.
- (d) No calculation is needed: 82% of animals leave alive, so Naïve Bayes will always predict that an animal leaves alive.

> **Answer — (a).** Score = P(class) × P(wild | class) × P(injured | class), with the class shares from Exhibit 1. Alive: 0.82 × 0.01 × 0.07 = 0.00057. Did not: 0.18 × 0.09 × 0.28 = 0.00454. The larger score wins: not alive. (b) leaves out the class shares. (c) The two scores are compared with each other, not with 0.5. (d) Strong evidence can outweigh the class share, as it does here.

Question 18 — Multiple choice (3 points)

Exhibit 6

```
# Take a new subset of the features, different from Exhibit 3. Hold out 30% for testing.
X = pd.get_dummies(shelter[['kind', 'sex', 'condition', 'intake_type', 'age_years']], dtype=int)
y = shelter['alive']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
```

Exhibit 7

```
# Trees of different depths: size and accuracy.
rows = []
for depth in [1, 2, 4, 6, 10, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train, y_train)
    rows.append({'max_depth': depth if depth else 'no limit', 'leaves': t.get_n_leaves(),
                 'train accuracy': round(accuracy_score(y_train, t.predict(X_train)), 3),
                 'test accuracy': round(accuracy_score(y_test, t.predict(X_test)), 3)})
pd.DataFrame(rows)
```

	max_depth	leaves	train accuracy	test accuracy
0	1	2	0.829	0.825
1	2	4	0.837	0.837
2	4	16	0.852	0.848
3	6	58	0.870	0.869
4	10	414	0.887	0.868
5	no limit	1695	0.910	0.856

The shelter wants the tree that will classify new animals correctly most often. Of the six trees in Exhibit 7, which `max_depth` should it use?

- (a) No limit: it has the highest training accuracy, so it has learned the most about the animals.
- (b) 6: it has the best test accuracy, and the deeper trees only fit the training animals better.
- (c) 1: its training and test accuracies are the closest, so it overfits the least.
- (d) 10: its training accuracy is higher than depth 6's and its test accuracy is about the same.

> **Answer — (b).** Test accuracy shows how often a tree is right on new animals, and it peaks at depth 6 (0.869). Deeper trees raise only the training accuracy: overfitting. (a) Training accuracy rewards memorizing. (c) Depth 1 underfits (0.825). (d) Same test accuracy, but 414 leaves instead of 58: prefer the simpler tree.

Question 19 — Short answer (4 points)

The shelter uses the depth-6 tree from Exhibit 7 (trained on the features in Exhibit 6) to decide which arriving animals get medical care and are sent to foster homes first. The tree was trained on animals that arrived from 2017 to 2024. In 2025 the shelter starts transferring every injured wild animal to a wildlife rehabilitation center on the day it arrives. A transfer counts as leaving the shelter alive. If the shelter keeps using the same tree, how will its mistakes on injured wild animals change, both in how often it is wrong and in the kind of mistake it makes? What should the shelter do about it? Answer in at most 4 sentences.

> **Answer.** The tree learned from 2017–2024, when wild and injured animals were far more common among animals that did not leave alive (Exhibits 4 and 5), so it likely flags injured wild animals as at risk. From 2025 they all leave alive, so the tree makes more mistakes on them, and the mistakes are false negatives (`alive` = 1 is the positive class): care and foster homes go to animals about to be transferred. The shelter should retrain on

> animals that arrived after the change (or handle these animals outside the model) and keep checking accuracy on recent animals.

Case B2 — NBA Rookies (use Handout B2)

A basketball team has the rookie-season statistics of more than 1,300 NBA players, in a DataFrame named `rookies`:

Column	Meaning
`GP`	Games played in the rookie season
`MIN`	Minutes played per game
`PTS`	Points scored per game
`REB`	Rebounds per game
`AST`	Assists per game
`TARGET_5Yrs`	1 if the player was still in the NBA 5 years after his rookie season, 0 if not

Every statistic is from the rookie season. Whether a player lasted 5 years is known only 5 years later. Right after a player's rookie season, the team wants to predict whether he will last 5 years, to decide whom to offer a long contract.

Exhibit 1

```
# Number of players, and the fraction who lasted 5 years or more.
print('Players:', len(rookies))
rookies['TARGET_5Yrs'].value_counts(normalize=True).round(3)
```

```
Players: 1328
TARGET_5Yrs
1    0.62
0    0.38
Name: proportion, dtype: float64
```

Exhibit 2

```
# The first few players.
rookies.head()
```

	GP	MIN	PTS	REB	AST	TARGET_5Yrs
0	36	27.4	7.4	4.1	1.9	0
1	35	26.9	7.2	2.4	3.7	0
2	74	15.3	5.2	2.2	1.0	0
3	58	11.6	5.7	1.9	0.8	1
4	48	11.5	4.5	2.5	0.3	1

Exhibit 3

```
# Features: the five rookie-season statistics. Hold out 30% of the players for testing.
X = rookies[['GP', 'MIN', 'PTS', 'REB', 'AST']]
y = rookies['TARGET_5Yrs']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
```

Question 20 — Multiple choice (2 points)

Before trusting any model with contract offers, the team wants a fair test on players the model has not seen. In Exhibit 3, the analyst splits the data with `stratify=y`. What does that do?

- (a) About 0.62 of the players in both the training set and the test set lasted 5 years.
- (b) About 0.3 of the players in both the training set and the test set lasted 5 years.
- (c) The training set and the test set each get 0.5 of the players.
- (d) The players who lasted 5 years (0.62) go to the training set and those who did not (0.38) to the test set.

> **Answer — (a).** `stratify=y` keeps the target's mix the same in both parts: 0.62 of all players lasted 5 years (Exhibit 1), so about 0.62 of the training players and of the test players did. (b) 0.3 is `test_size`, the share held out for testing. (c) The split is 0.7 / 0.3. (d) Both sets keep both classes.

Question 21 — Multiple choice (3 points)

Exhibit 4

```
# Logistic regression with the five features.
log_reg = LogisticRegression(max_iter=5000).fit(X_train, y_train)
pd.DataFrame({'coefficient': log_reg.coef_[0], 'exp(coefficient)': np.exp(log_reg.coef_[0])},
             index=X.columns).round(3)
```

	coefficient	exp(coefficient)
GP	0.040	1.041
MIN	-0.087	0.917
PTS	0.167	1.182
REB	0.329	1.390
AST	0.109	1.115

The scouts want to know which rookie statistics matter most when deciding whom to offer a long contract. Holding the other features fixed, which statement does `log_reg`, the logistic regression in Exhibit 4, support?

- (a) One more rebound per game makes a player more likely to last 5 years than one more point per game does.
- (b) One more minute per game makes a player more likely to last 5 years.
- (c) One more point per game raises the probability of lasting 5 years by 0.167.
- (d) One more assist per game makes a player less likely to last 5 years, because its coefficient is below 1.

> **Answer — (a).** One more rebound multiplies the odds by exp(0.329) = 1.390; one more point only by exp(0.167) = 1.182. (b) `MIN` has a negative coefficient (−0.087). (c) 0.167 is added to the log-odds, not to the probability. (d) `AST`'s coefficient is above 0 (exp 1.115, above 1), so assists raise the odds; the cutoff of 1 is for exp(coefficient).

Question 22 — Multiple choice (3 points)

Exhibit 5

```
# Scale to 0-1 with the training rows, then train two neural networks.
scaler = MinMaxScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

rows = []
for layers in [(5,), (100, 100)]:
    nn = MLPClassifier(hidden_layer_sizes=layers, solver='lbfgs', max_iter=2000,
random_state=1).fit(X_train_s, y_train)
    rows.append({'hidden layers': layers, 'training steps': nn.n_iter_,
                 'train accuracy': round(accuracy_score(y_train, nn.predict(X_train_s)), 3),
                 'test accuracy': round(accuracy_score(y_test, nn.predict(X_test_s)), 3)})
pd.DataFrame(rows)
```

	hidden layers	training steps	train accuracy	test accuracy
0	(5,)	46	0.727	0.672
1	(100, 100)	2000	0.897	0.622

In Exhibit 5 the team trains two neural networks on the scaled features: a small one with one hidden layer of 5 nodes, and a much bigger one with two hidden layers of 100 nodes, hoping the bigger one will predict better who will last 5 years. Which statement best describes the bigger network?

- (a) It fits the training players much better than new ones, and on new players it does about as well as predicting that every rookie lasts 5 years.
- (b) It is the better model, because it learned far more from the training players than the network with one hidden layer of 5 nodes.
- (c) It stopped too early; raising `max_iter` further would bring its test accuracy up to its training accuracy.
- (d) Its scaling was done wrong: the correct approach is to fit the scaler on all the rows, not only the training rows.

> **Answer — (a).** Training accuracy 0.897 against test 0.622: overfitting. Predicting "lasts" for everyone already scores about 0.62 (Exhibit 1), so the big network adds almost nothing on new players; the small one (test 0.672) does better. (b) Fitting the training players closely is not the goal. (c) It used all 2000 steps; more would fit the training data even more closely. (d) Fitting the scaler on the training rows only is correct; using all rows leaks test information.

Question 23 — Short answer (4 points)

Exhibit 6

```
# Logistic regression (standardized features) gives each test rookie a probability of lasting 5 years.
std = StandardScaler().fit(X_train)
log_reg_std = LogisticRegression(max_iter=5000).fit(std.transform(X_train), y_train)
prob = log_reg_std.predict_proba(std.transform(X_test))[:, 1]

# Offer a contract when the probability is at or above the threshold.
rows = []
for t in [0.3, 0.5, 0.6, 0.7, 0.8]:
    offer = (prob >= t).astype(int)
    rows.append({'threshold': t, 'contracts offered': offer.sum(),
                 'offered, did not last': ((offer == 1) & (y_test == 0)).sum(),
                 'not offered, did last': ((offer == 0) & (y_test == 1)).sum(),
                 'precision': round(precision_score(y_test, offer), 3),
                 'recall': round(recall_score(y_test, offer), 3)})
pd.DataFrame(rows)
```

	threshold	contracts offered	offered, did not last	not offered, did last	precision	recall
0	0.3	362	125	11	0.655	0.956
1	0.5	254	71	65	0.720	0.738
2	0.6	207	46	87	0.778	0.649
3	0.7	153	25	120	0.837	0.516
4	0.8	95	11	164	0.884	0.339

The team will offer a long contract to every player whose probability of lasting 5 years, as predicted by `log_reg_std` in Exhibit 6, is at or above a threshold. A long contract for a player who does not last wastes about $2 million. Not offering one to a player who does last costs the team about $1 million in lost value. Which threshold gives the team the lowest total cost on these test players? Give that total cost, and the percentage of the players who did last 5 years that the team misses at that threshold. Answer in at most 3 sentences.

> **Model answer — 0.7.** Cost = 2 × (offered, did not last) + 1 × (not offered, did last): 0.3 → $261M, 0.5 → $207M, 0.6 → $179M, 0.7 → 2 × 25 + 120 = $170M, 0.8 → $186M. At 0.7 the recall is 0.516, so the team misses 48.4% of the players who lasted (120 of 248).